

- Sistema decimal (0-9)
- Sistema binario (0, 1)

- Sistema octal (0-7)
- Sistema hexadecimal (0-F)

TABLAS

DECIMAL	BINARIO	OCTAL	BINARIO	HEXADECIMAL	BINARIO
0	0 0 0 0	0	0 0 0	0	0 0 0 0
1	0 0 0 1	1	0 0 1	1	0 0 0 1
2	0 0 1 0	2	0 1 0	2	0 0 1 0
3	0 0 1 1	3	0 1 1	3	0 0 1 1
4	0 1 0 0	4	1 0 0	4	0 1 0 0
5	0 1 0 1	5	1 0 1	5	0 1 0 1
6	0 1 1 0	6	1 1 0	6	0 1 1 0
7	0 1 1 1	7	1 1 1	7	0 1 1 1
8	1 0 0 0			8	1 0 0 0
9	1 0 0 1			9	1 0 0 1

SISTEMAS DE NUMERACIÓN

Potencias 2

$2^{10} \rightarrow 1024$
$2^9 \rightarrow 512$
$2^8 \rightarrow 256$
$2^7 \rightarrow 128$
$2^6 \rightarrow 64$
$2^5 \rightarrow 32$
$2^4 \rightarrow 16$
$2^3 \rightarrow 8$
$2^2 \rightarrow 4$
$2^1 \rightarrow 2$
$2^0 \rightarrow 1$
$2^{-1} \rightarrow 0.5$
$2^{-2} \rightarrow 0.25$
$2^{-3} \rightarrow 0.125$
$2^{-4} \rightarrow 0.0625$

Potencias 8

$8^7 \rightarrow 2097152$
$8^6 \rightarrow 262144$
$8^5 \rightarrow 32768$
$8^4 \rightarrow 4096$
$8^3 \rightarrow 512$
$8^2 \rightarrow 64$
$8^1 \rightarrow 8$
$8^0 \rightarrow 1$
$8^{-1} \rightarrow 0.125$
$8^{-2} \rightarrow 0.015625$
$8^{-3} \rightarrow 0.001953125$
$8^{-4} \rightarrow 0.0002441406$

Potencias 16

$16^{10} \rightarrow 1B \sim$
$16^9 \rightarrow 68M.M \sim$
$16^8 \rightarrow 4M.M \sim$
$16^7 \rightarrow 268435456$
$16^6 \rightarrow 16777216$
$16^5 \rightarrow 1048576$
$16^4 \rightarrow 65536$
$16^3 \rightarrow 4096$
$16^2 \rightarrow 256$
$16^1 \rightarrow 16$
$16^0 \rightarrow 1$
$16^{-1} \rightarrow 0.0625$
$16^{-2} \rightarrow 0.00390625$

¡Nota! - Indicar la base

Trabajamos con números expresados en diferentes bases, por ello es clave indicar en su subíndice la base, justo después del n°

$$1110_{(2)} \neq 1110_{(10)}$$

ESQUEMA DE CONVERSIONES ENTRE SIST. NUM.

BINARIO	A DECIMAL POR PESO	DE DECIMAL SUM DE PESOS	DIVISIONES / MULTIPLICACIONES base por la NUEVA BASE
	$1010_{(2)}$ $2^3 2^2 2^1 2^0$	Se resta la potencia de 2 que + se acerca por debajo	
OCTAL	POR PESO	DIVISIONES / MULTIPLICACIONES	A BINARIO
	$2374_{(8)}$ $2 \cdot 8^3 + 3 \cdot 8^2 + 7 \cdot 8^1 + 4 \cdot 8^0$	$1276 \overline{) 8}$ $159 \overline{) 8}$ 4 7 3 $\rightarrow 2374_{(8)}$	Hacemos grupos de 3 bits. DE OCTAL A BINARIO REPRESENTAR * Saberse la tablita.
HEXADECIMAL	POR PESO	DIVISIONES / MULTIPLICACIONES	A BINARIO
	$29C_{(16)}$ $2 \cdot 16^2 + 9 \cdot 16^1 + C \cdot 16^0$	$668 \overline{) 16}$ $41 \overline{) 16}$ 13 9 2 $\rightarrow 29C$	Igual que de octal a binario, solo que ahora grupos de cuatro y viceversa.

FORMATOS DE REPRESENTACIÓN

- FORMATOS DE CAMPO BINARIO (n° de bits manejados)
- Byte $\rightarrow 8$ bits
 - Palabra (Word) $\rightarrow 32$ bits
 - Half word $\rightarrow 16$ bits
 - Double word $\rightarrow 64$ bits
 - Extended word $\rightarrow 128$ bits
 - Nibble $\rightarrow 4$ bits

- FORMATOS DE CONVENIO BINARIO (Significado de bits)
- Formato representación entera
 - Formato complemento a 2
 - Formato como float IEEE 754
 - Formato alfanumérico ASCII
 - Mayúsculas.

Kilobyte (KB) 10^3 , Megabyte (MB) 10^6 , gigabyte (GB) 10^9 , Terabyte (TB) 10^{12} , Petabyte (PB) 10^{15}

- Bit de signo magnitud

El bit + significativo representa el signo (bit de signo), si es 0 $\rightarrow$ positivo (+), el resto de bits representa la magnitud. Si es 1 $\rightarrow$ negativo (-), el

- Complemento a la base restringida. (C1)

Si la base es 2, la base restringida es: $2-1=1$

El complemento a 1 de un número N_{10} de expresión binaria N_2 compuesto por n bits, se calcula:

$$C_{1N} = 2^n - N_{10} - 1 \rightarrow \text{Da el número en decimal}$$

NOTA: En binario es equivalente a intercambiar 1s y 0s. eg. $11_{(10)} = 1011_{(2)}$ $C_1 = 4_{(10)} = 0100$

⚠ En este caso todos los bits determinan la magnitud, ya que el BHS tiene peso negativo: -2^{n-1}

Por tanto, lo que hacemos es añadir un 110 a la izq del bit para evitar que es negativo/positivo. Es decir:

$11_{(10)} = 01011$ (5 bits)

$C_{11} = 2^5 - 11 - 1 = 32 - 12 = 20_{(10)} = 10100_{(2)}$

Bit signo

NOTA: sigue siendo intercambiar 0s por 1s y viceversa.

- Complemento a la base (C2)

Si la base es 2. Aquí ocurre lo mismo que anteriormente solo directamente le ponemos un bit de signo. La forma de calcularlo ahora es:

$$C_{2N} = 2^n - N_{10} \rightarrow \text{se el más uno}$$

NOTA: En binario es equivalente a cambiar ceros por unos y unos por ceros.

ESQUEMA DE CONVERSIONES ENTRE SIST. NÚM.

T2.2⁰
REPRESENTACIÓN
DE LA
INFORMACIÓN
Y
SISTEMAS

BINARIO	A DECIMAL POR PESO	DE DECIMAL SUM DE PESOS	DIVISIONES / MULTIPLICACIONES base POR LA NUEVA BASE
	10 10 ₍₂₎ 2 ³ 2 ² 2 ¹ 2 ⁰	Se resta la potencia de 2 que + se acabe por debajo	base 25 $\begin{array}{r} 12 \\ 12 \\ 12 \\ 12 \\ 12 \end{array}$... 0's 0 = 1's 0's 0 = 1's
OCTAL	POR PESO	DIVISIONES / MULTIPLICACIONES	
	2374 ₍₈₎ 2 · 8 ³ + 3 · 8 ² + 7 · 8 ¹ + 4 · 8 ⁰	127618 4	10 de 3 bits. 10 REPRESENTAR lita.
HEXADECIMAL	POR PESO	DIVISIONES / MULTIPLICACIONES	
	29C ₍₁₆₎ 2 · 16 ² + 9 · 16 ¹ + C · 16 ⁰	DI 66 4	10 igual a ahora y viceversa.

(2) → (10) → (pesos)
(10) → (2) (divisiones)
(8) → (10) (pesos)
(10) → (8) (divisiones) / (Tabla)
(8) → (2) (Tabla)
(16) → (10) (divisiones)
(10) → (16) (divisiones)
(16) → (2) (Tabla)

FORMAS DE REPRESENTACIÓN

- FORMAS DE CAMPO BINARIO (n° de bits manejados)
 - Byte → 8 bits
 - Palabra (word) → 32 bits
 - Half word → 16 bits
 - Double word → 64 bits
 - Extended word → 128 bits
 - Nibble → 4 bits

Kilobyte (KB) 10^3 , Megabyte (MB) 10^6 , gigabyte (GB) 10^9 , Terabyte (TB) 10^{12} , Petabyte (PB) 10^{15}

- Bit de signo magnitud

El bit + significativo representa el signo (bit de signo), si es 0 → positivo (+), el resto de bits representa la magnitud. Si es 1 → negativo (-), el resto de bits representa la magnitud.

- Complemento a la base restringida. (C1)

Si la base es 2, la base restringida es: $2-1=1$

El complemento a 1 de un número N_{10} de expresión binaria N_2 compuesta por n bits, se calcula:

$$C_{1N} = 2^n - N_{10} - 1 \rightarrow \text{Da el número a dudar}$$

Nota: En binario es equivalente a intercambiar 1s y 0s. eg. $11_{10} = 1011_2$, $C_1 4_{10} = 0100_2$

⚠ En este caso todos los bits determinan la magnitud, ya que el DMS tiene peso negativo: -2^{n-1}

Por tanto, lo que hacemos es añadir un 1/0 a la izq del bit para indicar que es negativo/positivo. Es decir:

$11_{10} = 01011_2$ (5 bits)
↓
bit signo
 $C_{111} = 2^5 - 11 - 1 = 32 - 12 = 20_{10} = 10100_2$

Nota: sigue siendo intercambiar 0s por 1s y viceversa.

- Complemento a la base (C2)

Si la base es 2. Aquí ocurre lo mismo que anteriormente pero directamente la ponemos un bit de signo. La forma de calcularla ahora es:

$$C_{2N} = 2^n - N_{10} \rightarrow \text{se el más uno}$$

Nota: En binario es equivalente a cambiar unos por ceros y ceros por unos al menos significativo.

T2.2
REPRESENTACIÓN
NUMÉRICA

Formato Exceso M

El valor de M se obtiene del 0/10 y en la columna binaria se obtiene el valor de M en Código Binario Natural.

E.g. Exceso 3

DECIMAL	Exceso 3
-3	000
-2	001
-1	010
0	011
1	100
2	101
3	110
4	111

• Si el BHS es 0 entonces la cantidad es 0 o negativa. Si es 1 es +.

• El valor + alto es todo 1 y equivale a $M+1_{(10)}$

• El valor + bajo es todo 0 y equivale a $-M_{(10)}$

EXCESO 127

DECIMAL	EXCESO 127
-127	0000000
-126	0000001
...	
-1	0111110
0	0111111
1	1000000
...	
126	1111110
127	1111111
128	1111111

Números reales en coma flotante

Representación basada en la notación científica, para representar números muy pequeños o grandes.

Partes de coma flotante

- Signo: indica si el número es positivo o negativo
- Mantisa: representa la magnitud del número
- Exponente: indica el desplazamiento de la coma fraccionaria

$$(-1)^{\text{Signo}} * \text{MANTISA} * \text{BASE}^{\text{Exponente}}$$

Para normalizar los números a coma flotante, se mueve esta hasta que la parte entera del número valga cero:

→ Se reduce la cantidad de información almacenar, ya que la parte entera es siempre 0 (de un número normalizado).

$$\text{E.g. } 241506800_{(10)} = 0.2415068 \cdot 10^9$$

signo: +
mantisa: 2415068
exponente: 9

Números binarios en coma flotante.

Para representar números binarios en coma flotante se usa el estándar IEEE 754 (2008).

Lo Formato a simple precisión, usa 32 bits (1 palabra) para representar.

SIGNO 1 bit	EXPONENTE 8 bits	MANTISA 23 bits
----------------	---------------------	--------------------

(Existe el formato de doble precisión que emplea 64 bits)

Lo Mantisa en simple precisión.

La mantisa de un número en coma flotante se normaliza para que exista una única representación válida. Normalizar es mover la coma hasta que la parte entera sea 1, por lo que se puede omitir ese bit (se asume implícitamente y ahora equivale a 1).

$$\text{E.g. } 0.0000101_{(2)} = 0.101 \cdot 2^{-4} = 1.01 \cdot 2^{-5}$$

Lo Exponente en simple precisión

Para el exponente se usa el formato a exceso, donde sumamos un desplazamiento de $(2^{n-1} - 1)$ al valor real del número, para poder guardarlo como positivo. (n es el número de bits).

T2.2

REPRESENTACIÓN NUMÉRICA

Formato Exceso M

El valor de M se obtiene del 0/10 y en la columna binaria se obtiene el valor de M en Código Binario Natural.

E.g. Exceso 3

DECIMAL	Exceso 3
-3	000
-2	001
-1	010
0	011 → 3
1	100
2	101
3	110
4	111

- Si el BMS es 0 entonces la cantidad es 0 o negativa. Si es 1 es +.
- El valor + alto es todo 1 y equivale a $M+1$ (10)
- El valor + bajo es todo 0 y equivale a $-M$ (10)

Exceso 127

DECIMAL	Exceso 127
-127	0 00 0000
-126	0 00 0001
...	
-1	0 1111 1110
0	0 1111 1111
1	1 0000 0000
...	
126	1 1111 1101
127	1 1111 1110
128	1 1111 1111

Números reales en coma flotante

Representación basada en la notación científica, para representar números muy pequeños o grandes.

- Partes de coma flotante
- Signo: indica si el número es positivo o negativo
 - Mantisa: representa la magnitud del número
 - Exponente: indica el desplazamiento de la coma decimal

$$(-1)^{\text{Signo}} * \text{MANTISA} * \text{BASE}^{\text{Exponente}}$$

Para normalizar los números a coma flotante, se mueve esta hasta que la parte entera del número valga cero:

- Se reduce la cantidad de información a almacenar, ya que la parte entera es siempre 0 (de un número normalizado).

E.g. $241506800_{(10)} = 0.2415068 \cdot 10^9$

signo: +
mantisa: 2415068
exponente: 9

Números binarios en coma flotante

Para representar números binarios en coma flotante

Lo Formato a simple precisión, usa 32 bits (1 pa

SIGNO	EXPONENTE	MANTISA
1 bit	8 bits	23 bits

La Mantisa en simple precisión.

La mantisa de un número en coma flotante una única representación válida. Normalizar entera sea 1, por lo que se puede omitir es

E.g. $0.0000101_{(2)} = 0.101 \cdot 2^{-4} = 1.01 \cdot 2^{-5}$

Lo Exponente en simple precisión

Para el exponente se usa el formato a exceso, donde sumamos un desplazamiento de $(2^{n-1} - 1)$ al valor real del número, para poder guardarlo como positivo. (n es el número de bits).

SIGNO	EXP	MANTISA
1	8	23

IEEE 754 - Sistema

- n° positivo a binario (signo y se sabe)
- Normalizar a binario
- La exponencia sumará 127
- Ya podemos representar en IEEE 754

Sistema - IEEE 754

- binario → normalizado → binario
- binario a decimal
- Exponente = 127

C.g. Si tuviéramos un exponente de 3 bits $\rightarrow$ representaríamos en exceso 3. ($2^{3-1} - 1 = 3$)

T2.3
REPRESENTACIÓN
DE NÚMEROS
REALES EN
CUM
FLUJANTE

DEC	EXCESO
-3	000
-2	001
-1	010
0	011
1	100
2	101
3	110
4	111 $\rightarrow$ RESERVADO

CON este formato los exponentes positivos tendrán un 1 (BMS) y los negativos un 0 (BMS).

En el formato IEEE754 se usa el exceso 127 ($2^{8-1} - 1$) $\rightarrow$ puesto que el exponente tiene 8 bits. El exponente irá de -127 a 128.

CASOS ESPECIALES EN SIMPLE PRECISIÓN (exceso 127)

El valor -127 (00000000)

- Lo si exponente = 0 y mantisa = 0 $\rightarrow$ se está representando el 0
 - Lo si " " y mantisa $\neq$ 0 $\rightarrow$ número no normalizado. Para representar números muy pequeños cercanos al cero. El bit implícito no es 1 sino 0 y exponente -126.
- $\rightarrow (-1)^{\text{signo}} \times 0^{\text{Mantisa}} \times 2^{-126} \rightarrow$ valor del número no normalizado.

El valor 128 (11111111)

- Lo si mantisa = 0 $\rightarrow$ se está representando el infinito
- Lo si mantisa $\neq$ 0 $\rightarrow$ se representa NaN (Not a Number) $\rightarrow \sqrt{-1}, \frac{0}{0}, \frac{\infty}{\infty}$, etc.

RESOLVER EJERCICIOS

TRADUCIR DE SISTEMA NUMÉRICO AL IEEE 754

- 1- Pasar no positivo a binario
- 2- Normalizarlo a binario (implícito)
- 3- Pasar al IEEE 754 $\rightarrow (-1)^{\text{signo}} \times \text{mantisa} \times \text{base}$

DE IEEE 754 A SISTEMA

1.

1	8	mantisa
signo	exponente	32
2. Binario normalizado $\rightarrow$ binario
3. binario a decimal

CÓDIGOS BINARIOS PARA NÚMEROS DECIMALES

Código BCD 8421 (BCD Natural)

- * BCD 8421 versión usada de los BCD
- * Cada dígito decimal se representa con 4 bits, pero nos sobran algunos, ya que $2^4 = 16$.
- * Ponderado: indica los pesos de cada uno de los 4 bits:
 $2^3 \ 2^2 \ 2^1 \ 2^0 \rightarrow 8421$

* Equivalencia BCD 8421 es:

DECIMAL A BCD

25718

0011 0101 0111 0001 1000

BCD A DECIMAL

- 1- Comienza por la derecha
- 2- Grupos de 4 bits
- 3- sustituir por dígito correspondiente.

DEC	BCD natural
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
	1010
	1011
	1100
	1101
	1110
	1111

no válidos

2.4
CÓDIGOS
BINARIOS

Código Aiken : BCD 2421

- * código ponderado como el anterior : 2421

- * Autocompletado : cambiando 0's por 1's obtenemos el complemento a 9 del correspondiente.

0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

no válidos	0101
	0110
	0111
	1000
	1001
	1010

Código BCD Exceso 3

↳ No ponderado.

↳ Cada palabra se obtiene sumando 3 a la palabra correspondiente en BCD 8421.

↳ Autocompletado: 0's y 1's se obtiene el complemento a 9 del correspondiente.

No. valores	0000	0001	0010
0	0011	→ 3	
1	0100	→ 4	
2	0101	→ 5	
3	0110	→ 6	
4	0111	→ 7	
5	1000	→ 8	
6	1001	→ 9	
7	1010	→ 10	
8	1011	→ 11	
9	1100	→ 12	
	1101		
	1110		
	1111		

Código Biquinario

↳ No ponderado de 7 bits

↳ Si 1º bit es 1 está entre 5-9
si es 0 está entre 0-4

↳ El resto es un número del rango.

VANTAJA → permite detectar errores en un solo bit
DESVENTAJA → hace falta 7 bits para representar lo que antes hacíamos con 4.

0100001	0
0100010	1
0100100	2
0101000	3
0110000	4
1000001	5
1000010	6
1000100	7
1001000	8
1010000	9

01 ↙
10 ↘

Código Gray

↳ Código no ponderado y no aritmético.

↳ Código que puede tener cualquier n° de bits

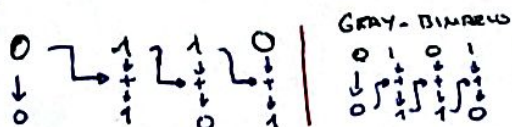
↳ Continuo: Solo varía un bit desde una palabra de

↳ Cíclico: Continuidad entre la primera y última

• Conversión BINARIO GRAY

1. BMS queda igual

2. De izquierda a derecha se suma cada par adyacente de bits para obtener el código Gray (se descarta acarreo)



0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

Código Johnson

↳ Como el Gray

- ↳ no ponderado
- ↳ no aritmético
- ↳ continuo
- ↳ cíclico

↳ Con n bits se puede codificar 2^n valores (cualquier n° de bits)

0	00000
1	00001
2	00011
3	00111
4	01111
5	11111
6	11110
7	11000
8	10000
9	

CÓDIGOS ALFANUMÉRICOS

T2.4

CÓDIGOS

BINARIOS

- Código ASCII (American Standard Code for Information Interchange) (Hexadecimal)
 - 7 bits $\rightarrow$ 128 codificaciones (150 646)
 - 8 bits $\rightarrow$ Respete las 128 codificaciones y añade 128 para otros lenguajes:
 - $\hookrightarrow$ ISO 8859 - n $\rightarrow$ n identifica el juego de caracteres según el lenguaje
 - $\hookrightarrow$ ISO 8859 - 1 $\rightarrow$ Latin-1 (acentos, tildes, diéresis...)
 - $\hookrightarrow$ ISO 8859 - 8 $\rightarrow$ caracteres griegos.
- Código EBCDIC (Extended Binary Code Decimal Interchange)
 - Representa 256 caracteres de 8 bits $\rightarrow$ va más allá de los dígitos decimales del BCD.
 - Poco usado en ordenadores por la existencia de ASCII.

PERO NINGUNO DE LOS DOS CUBRIRÁ TODOS LOS CARACTERES:

• UNICODE (Hexadecimal)

Basado en ASCII, busca albergar todos ser de caracteres existentes (incluso los más antiguos, como los símbolos egipcios).

$\hookrightarrow$ Facilita la manipulación de textos plurilingües

CARACTERÍSTICAS

Cada carácter posee:

- Nombre: LATIN CAPITAL LETTER A
- Identificador: U + n hexadecimal
- Info extra: alfabeto, sist de escritura, categoría...

FORMATOS DE CODIFICACIÓN

UTF-8

UTF-16

UTF-32

BMP $\rightarrow$ alberga los caracteres

$\leftarrow$ y siguientes

- Códigos para la detección y corrección de errores.

Hay códigos detectores que permiten detectar errores y códigos correctores que permiten corregir errores.

CONCEPTO DE N-CUBO

Un n-cubo binario es un cubo con un total de 2^n vértices, cada uno de los cuales se corresponde con una combinación particular de n bits.

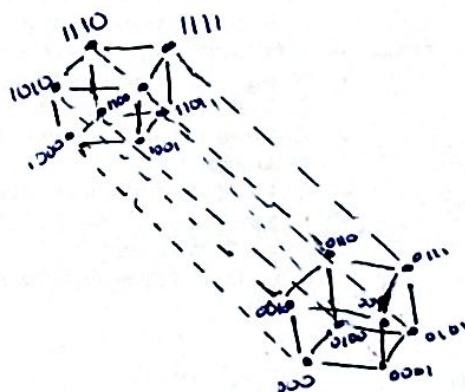
Dos vértices están unidos por una arista si y solo si las combinaciones de bits correspondientes difieren en un único bit.

C.g.

0 — 1 n = 1

10 — 11
00 — 01 n = 2

101 — 111
100 — 110
001 — 011
010 n = 3



T2.5

CÓDIGOS

DETECCIÓN

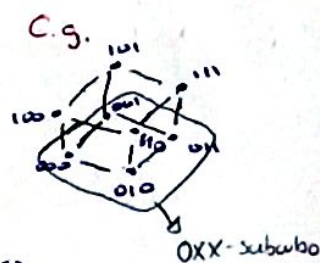
Y

CORRECCIÓN

DE

ERRORES

- Dentro de un n -cubo puede haber varios m -subcubos.
- Los m bits tomarán todas las 2^m posibles combinaciones.
 - Los restantes $n-m$ bits siempre tienen el mismo valor



El subcubo se especifica con una combinación de n bits que contiene los $n-m$ bits fijos y una x en las restantes m posiciones.

- Distancia booleana o de Hamming.

La distancia entre 2 vértices es = al número de pasos del camino + corto entre ellos. (número de bits que diferencian a los 2 vértices)

- Entre 2 vértices adyacentes siempre es 1
- 2 vértices no adyacentes en un 2-subcubo están a una distancia 2.

CÓDIGO DE DETECCIÓN DE ERRORES

Realmente los n -cubos son códigos de n bits con todas las combinaciones de esos n bits.

- Esas combinaciones son las palabras del código
- Las combinaciones no incluidas no son palabras del código; son errores.

- Detección de errores en un bit

Para detectar errores en un solo bit es necesario que la distancia mínima entre 2 palabras del código sea 2. (no puede corresponder a vértices adyacentes).

- Bit de paridad

Para mantener esta distancia de 2 en cualquier código es necesario usar la mitad de las combinaciones, $2^n/2 = 2^{n-1}$

Esto implica que se ha de usar $n+1$ bits para codificar datos:

→ n bits de info y 1 bit de paridad.

- El bit de paridad puede tomar un valor tal que las mas de las combinaciones sea par si son del código y si no pues impar, o viceversa. Depende si el código de paridad es par (combinaciones con 1s par) o impar (1s impar)

E.g.



Código de paridad impar
" " " par.

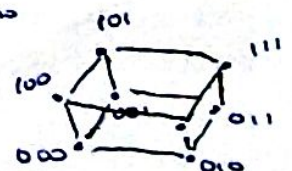
Nota: se debe especificar la ubi del bit de paridad.

Sin embargo, no permite corregir y solo puede detectar 1 error.

CÓDIGO DE CORRECCIÓN DE ERRORES

Para poder corregir errores es necesario aumentar la distancia entre palabras a 3. Aunque tmb nos permite hallar errores en 2 bits. Pero no los 6 casos a la vez.

Nota: que la distancia 3 de un código es su C_1 .



CL MÉTODO DE HAMMING / CORREGIR ERRORES CON DISTANCIA MÍNIMA DE 3

Por cada valor entero m hay un código Hamming con $2^m - 1$ bits que contienen:

- ↳ m bits de paridad
- ↳ $2^m - 1 - m$ bits de información.

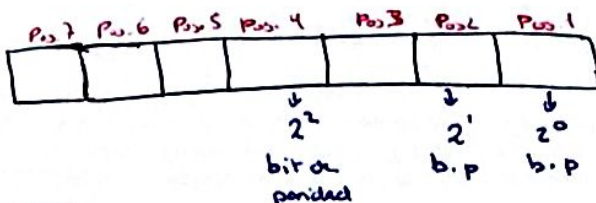
Ej. $m=3 \rightarrow$ código (7, 4)

$$2^3 - 1 = 7 \rightarrow 7 \text{ bits en total}$$

$$2^3 - 1 - 3 = 4 \rightarrow 4 \text{ bits de información (3 de paridad)}.$$

Los bits de paridad están entremezclados. Si numeramos las posiciones de los bits de 1 hasta $2^m - 1$ podemos decir que hay 1 bit de paridad en la pos 2^k (0 ≤ k ≤ m-1)

$m=3$
 $k=0$
 $k=1$
 $k=2$



Nota: el 2^k se refiere al número de la posición. Por ejemplo, $2^2 = 4$ pues hay un bit de paridad en la posición 4.

CÁLCULO DE PARIDAD

El bit de paridad en la posición 2^k le corresponderá todos los bits situados en una posición cuya representación binaria tenga un 1 en la posición 2^k . 0 sea, si hay un bit de paridad en 2 entonces todos los que tengan un bit en binario en esa posición 2 les afecta este bit.

Ej. 1011 → tiene 4 de info, ergo 3 de paridad (7 bits)

1	0	1		1		
111	110	101	100	011	010	001

Bit 1

Bit 2

Bit 4

Ergo: el código Hamming de paridad es 1010101.

CORRECCIÓN DE UN ERROR DE BIT

Se calcula la paridad de cada grupo y la suma de las posiciones de los bits de paridad incorrectos nos dará la posición del bit incorrecto.

Ej. → Recibimos la palabra 111 0101 en sistema paridad par.

1	1	1	0	1	0	1

↓
ERROR bit 2

Bit 1 ✓ → correcto por

Bit 2 x impo

Bit 4 x impo

Hay un error en la posición 2. El bit debe ser 0

CORREGIR Y DETECTAR ERRORES SIMULTÁNEAMENTE

Para corregir x errores de un bit se necesita una distancia mínima de $2x + 1$

Un código con la distancia mínima ($2x + y + 1$) será capaz de:

- ↳ Corregir x errores
- ↳ Detectar hasta y errores.

OPERACIONES ARITMÉTICAS BÁSICAS

• Suma binaria

$$\begin{aligned} 0+0 &= 00 \rightarrow \text{Suma } 0, \text{ acarreo } 0 \\ 0+1 &= 01 \rightarrow \text{Suma } 1, \text{ acarreo } 0 \\ 1+0 &= 01 \rightarrow \text{Suma } 1, \text{ acarreo } 0 \\ 1+1 &= 10 \rightarrow \text{Suma } 0, \text{ acarreo } 1 \end{aligned}$$

• Resta binaria

$$\begin{aligned} 0-0 &= 00 \rightarrow \text{Diferencia } 0, \text{ préstamo } 0 \\ 0-1 &= 11 \rightarrow \text{Diferencia } 1, \text{ préstamo } 1 \\ 1-0 &= 10 \rightarrow \text{Diferencia } 1, \text{ préstamo } 0 \\ 1-1 &= 00 \rightarrow \text{Diferencia } 0, \text{ préstamo } 0 \end{aligned}$$

T3.1

ARITMÉTICA
BINARIA

• Multiplicación

$$\begin{aligned} 0 \times 0 &= 0 \\ 0 \times 1 &= 0 \\ 1 \times 0 &= 0 \\ 1 \times 1 &= 1 \end{aligned}$$

• División

sigue el procedimiento de multiplicación y resta.

$$\begin{array}{r} 110 \quad 111 \\ -11 \quad 10 \\ \hline 000 \\ -00 \\ \hline 00 \end{array} \quad \begin{array}{r} 6 \quad 13 \\ -9 \quad 2 \end{array}$$

✶ No son importantes.

OPERACIONES CON NÚMEROS ENTEROS

Los sistemas digitales deben poder representar números positivos y negativos. Vamos a ver 3 formatos binarios para representar números enteros, donde en todas el BMS representa el signo y tiene una posición fija.

En números sin signo se puede producir acarreo, es decir que el resultado se salga del rango de representación.

$$\begin{array}{r} 11111111 \rightarrow 255 \\ 00000011 \rightarrow 3 \\ \hline 10000010 \rightarrow 258 \end{array}$$

Lo sobrepasamos de rango.

• Signo-magnitud

Lo Bit + a la izquierda (BMS) representa el signo $\begin{cases} 1 \rightarrow \text{negativo} \\ 0 \rightarrow \text{positivo} \end{cases}$, el resto representan la magnitud del número.

$$\begin{array}{r} 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0 \\ 0 \mid 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \rightarrow 21 \\ 1 \mid 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \rightarrow -21 \end{array}$$

Nota: Los n en signo-magnitud de tamaño n bits representan valores desde $-(2^{n-1}-1)$ hasta $(2^{n-1}-1)$.
 n es número de bits.

Ergo: si tengo 8 bits $2^7-1 = 127$

Así que pueden representar desde -127 hasta 127 .

SUMA Y RESTA

→ Se comparan signo y magnitud

→ si signo = se mantiene el signo y se suma la magnitud.

→ si el signo de los operandos es distinto, se resta la magnitud mayor menos la menor y se pone el signo de la mayor.

→ Multiplicación y división: si signo = entonces +, else -.

Complemento a 1.

El Complemento a 1 se obtiene cambiando los 1 por 0 y viceversa.

Lo Los números positivos se representan igual que n° positivos en signo positivo.

Lo Los números negativos son el complemento a 1 del n° positivo.

Ej.

$$\begin{array}{ccccccc} 0 & 0 & 0 & 1 & 0 & 0 & 1 \rightarrow 21 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 \rightarrow -21 \end{array}$$

El valor de los n° negativos se calcula asignando el valor negativo de BMS y sumando 1 al resultado

Ej.

$$\begin{array}{cccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ -128 & + & 64 & + & 32 & + & 16 & + & 8 & + & 4 & + & 2 & \rightarrow & -22 & + & 1 & \rightarrow & -21 \end{array}$$

NOTA: REPRESENTA LA MISMA CANTIDAD QUE EL SIGNO MÁXIMO $-(2^{n-1}-1)$ hasta $+(2^{n-1}-1)$.

RESTA

La resta es innecesaria, pues podemos hacer el C1 y sumar

Al sumar se puede producir acarreo a el último bit. Si esto ocurre, se suma al resultado de la suma

El resultado de una suma será incorrecto cuando al sumar 2 números positivos de un resultado negativo (si el BMS es 1).

$$\begin{array}{r} 00011001 \\ + 1100111 \\ \hline 100000000 \\ \text{ACARREO} \rightarrow 1 \\ \hline 00000001 \end{array}$$

Complemento a 2

Se obtiene sumando 1 al C1 de un número.

$$\begin{array}{ccccccc} 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \rightarrow 21 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \rightarrow -21 \end{array}$$

El valor de los n° negativos se calcula como antes (con el C1). El BMS es el que representa el signo.

NOTA: Este es capaz de representar desde $-(2^{n-1})$ hasta $+(2^{n-1}-1)$.

SUMA - OVERFLOW

Si se suma 2 valores del mismo signo el resultado debe ser del mismo signo.

Si se produce overflow (distinto signo) entonces es incorrecto pq necesita + bits para representar

ACARREO en SUMA

Se descarta el acarreo y ya.

$$\begin{array}{r} 11111011 \rightarrow -5 \\ + 11111010 \rightarrow -6 \\ \hline \otimes 11110101 \rightarrow -11 \end{array}$$

$$\begin{array}{r} 01111101 \rightarrow 125 \\ + 00111010 \rightarrow 58 \\ \hline 10110111 \rightarrow 183 \end{array} \quad \begin{array}{l} \text{ma'1} \\ 125 \\ 58 \\ \hline 183 \end{array}$$

DIVISIÓN $\rightarrow$ SE HACE CON SUMAS DEL COMPLEMENTO A 2. ($\neq$ signo +) $\neq$ signo -).

FORMULAS DE LOS COMPLEMENTOS

C1

$$C1_n = 2^n - N_{10} - 1 \rightarrow \text{Lo da en decimal}$$

C2

$$C2_n = 2^n - N_{10} \rightarrow \text{Lo da en decimal}$$

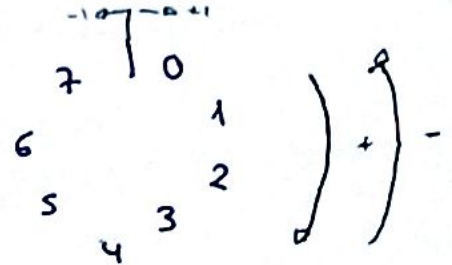
ARITMÉTICA OCTAL

- Suma octal

$$\begin{array}{r} 1 4 1 6 3 \\ + 7 5 2 0 \\ \hline 1 3 7 0 3 \end{array}$$

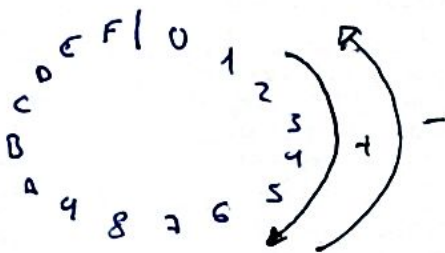
+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	10
2	2	3	4	5	6	7	10	11
3	3	4	5	6	7	10	11	12
4	4	5	6	7	10	11	12	13
5	5	6	7	10	11	12	13	14
6	6	7	10	11	12	13	14	15
7	7	10	11	12	13	14	15	16

POCO PRÁCTICO



ARITMÉTICA HEXADESIMA

- Suma Hexadecimal



$$\begin{array}{r} 1 1 \\ 6 A E \\ + 1 F A \\ \hline 8 A 8 \end{array}$$

RESIDUOS OCTAL (TMD SE PUEDE HEX)

$$\begin{array}{r} 5278 \\ - 365 \\ \hline 142 \end{array}$$

777 - 365 = 422

$$\begin{array}{r} 527 \\ + 422 \\ \hline 142 \end{array}$$

SE SUMA EL ACARRO

Para resolver la resta de números y representar negativos

- ↳ Bit de signo - magnitud
- ↳ Complemento a 9
- ↳ Complemento a 10

- Bit de signo - magnitud

↳ El signo viene dado por un grupo adicional de 4 bits

↳ Se usa cualquier cadena que no se use en el código para definir si es + o -.

Para definir + o -

valor	valor
1010	+
1011	+
1000	+
1101	-
1111	-

- Complemento a 9.

Básicamente complemento a la base restringida: $(C_9 = 10^n - N_{10} - 1)$

$$210 \rightarrow 1000 - 210 - 1 = 789$$

Y TMD conviene un bit de signo.

$$\begin{array}{r} 210 \rightarrow 210 \quad 0 \quad 0010 \quad 0001 \quad 0000 \\ -210 \rightarrow 789 \quad 1 \quad 0111 \quad 1000 \quad 1001 \end{array}$$

Nota: el complemento a 9 es como el complemento a 2 pero este se hace a base de 10 (n es el n° de dígitos dec)

- Complemento a 10

Lo mismo que la anterior incluida la Nota. $(C_{10} = 10^n - N_{10})$

$$214 \rightarrow 1000 - 214 = 789$$

$$\begin{array}{r} 214 \rightarrow 214 \quad 0 \quad 0010 \quad 0001 \quad 0000 \\ -210 \rightarrow 789 \quad 1 \quad 0111 \quad 1000 \quad 1001 \end{array}$$

T3.2

ARITMÉTICA DE BASES

T3.3

ARITMÉTICA EN BCD

• SUMA BCD 8421

1. Se realiza díg. a díg. una suma binaria de 4.
2. Si se genera acarreo + se propaga a la suma sig.
3. Si el resultado $\leq 9 \rightarrow$ es válido
4. Si el resultado $> 9 \rightarrow$ se suma 6 (binario) al resultado $\rightarrow$ se genera un acarreo

T3.

ARITMÉTICA
BCD

C. 9.

$$23 + 15 = 38$$

$$16 + 15 = 31$$

$$\begin{array}{r} 0001 \ 0110 \ 16 \\ + 0001 \ 0101 \ 15 \\ \hline 0010 \ 1011 \rightarrow 23 \\ \quad 1 \ 0110 \\ \hline 0011 \ 0001 \rightarrow 31 \end{array}$$

$$\begin{array}{r} 0010 \quad 0011 \rightarrow 23 \\ + 0001 \quad 0101 \rightarrow 15 \\ \hline 0011 \quad 1000 \rightarrow 38 \end{array}$$

1. Se suma 4 bits
2. Si el resultado > 9 (1001) $\rightarrow$ acarreo Suma 6
3. Si genera un nuevo acarreo no suma 6, se sumas con el otro 3 ya

• RESTA BCD 8421

Se pasa a complemento a 10 el n° a restar $\rightarrow 237 - 235 = 237 + (-235)$
Cio = 765

$$\begin{array}{r} 0 \quad 0011 \quad 0010 \quad 0111 \\ + 1 \quad 0111 \quad 0110 \quad 0101 \\ \hline 1 \ 0100 \quad 1 \ 0000 \quad 1000 \\ + 0110 \quad 1 \quad 0110 \\ \hline ① \ 0000 \quad 1001 \quad 0010 \end{array}$$

Se descarta acarreo

Según base $\left\{ \begin{array}{l} \rightarrow \text{Complemento a la base restringida} \\ \rightarrow \text{Complemento a base} \end{array} \right.$

$$C_b = b^n - N_{10} - 1$$

$$C_b = b^n - N_{10}$$

RECORDATORIO
BASES
1
COMPLEMENTO

$$\left. \begin{array}{l} C_2 \\ C_1 \end{array} \right\} \begin{array}{l} \rightarrow C_1 \rightarrow C_2 = 2^n - N_{10} - 1 \\ \rightarrow C_2 \rightarrow C_1 = 2^n - N_{10} \end{array}$$

$n \equiv$ n° de bit de N_{10} a binario

$$\left. \begin{array}{l} C_n \\ C_{10} \end{array} \right\} \begin{array}{l} C_n \rightarrow C_n = 10^n - N_{10} - 1 \\ C_{10} \rightarrow C_{10} = 10^n - N_{10} \end{array}$$

$n \equiv$ n° de dígitos decimales.

RESUMEN DE ARITHÉTICAS

OPERACIONES ARITHÉTICAS BÁSICAS

SUMA

$$\begin{aligned} 0 + 0 &= 00 \\ 0 + 1 &= 01 \\ 1 + 0 &= 01 \\ 1 + 1 &= 10 \end{aligned}$$

RESTA

$$\begin{aligned} 0 - 0 &= 00 \\ 0 - 1 &= 11 \rightarrow \text{prestamo 1} \\ 1 - 0 &= 10 \\ 1 - 1 &= 00 \end{aligned}$$

MULTIPLICACIÓN

$$\begin{aligned} 0 \times 0 &= 0 \\ 0 \times 1 &= 0 \\ 1 \times 0 &= 0 \\ 1 \times 1 &= 1 \end{aligned}$$

DIVISIÓN

Es igual que la división normal con algoritmo de x y $-$.

OPERACIONES NÚMEROS ENTEROS

• Signo - magnitud.

- ↳ Desbordamiento (overflow): que el resultado se salga de rango.
- ↳ Acarreo: bit de + que indica el overflow

El bit + a la izquierda representa el signo ($0 \rightarrow +$, $1 \rightarrow -$) el resto la magnitud del n°.

$$\begin{array}{c|ccccccc} 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \end{array} \rightarrow 16 + 4 + 1 = 21$$

• Suma/resta:

- Se compara signo y magnitud de los operandos.
- Si signo = se + magnitudes y mantiene el signo.
- Si signo $\neq$ se - la magnitud mayor menos la menor y signo de la mayor.

• Multiplicación/división:

- Multiplica/divide. Si signo = $\rightarrow +$, si $\neq \rightarrow -$.

• Complemento a 1.

Se obtiene cambiando todos los 0 por 1 y viceversa. (Representa el n° negativo).

$$\begin{array}{c|ccccccc} -2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \end{array} \rightarrow 16 + 4 + 1 = 21$$

$$\begin{array}{c|ccccccc} 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \end{array} \rightarrow -128 + 64 + 32 + 8 + 2 \rightarrow -22 \rightarrow -21$$

- ↳ Resta: innecesaria, para restar pasamos a complemento a 1 y sumamos
Ej: $24 - 25 = 24 + (-25)$

- ↳ Carry: si se produce se descarta y se suma al resultado de la suma.
- ↳ Desbordamiento: se identifica si un resultado no tiene el signo que debía.

• Complemento a 2.

Es = que complemento a 1, solo que se le suma 1.

$$\begin{array}{c|ccccccc} 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \end{array} \left. \begin{array}{l} \text{se mantiene} \\ \text{se suma 1} \end{array} \right\} \rightarrow 21$$

- ↳ Overflow: es igual que complemento a 1
- ↳ Carry: no se suma, se descarta.

NOTA: Si sumados tienen distinto signo es imposible un desbordamiento.

NOTA: Si se tiene n° en otras bases $\rightarrow$ pasamos a binario y luego complemento.

• Multiplicación: como toda la vida.

• División:

• Si operandos $\neq 0$ resultado $+$, $\neq$ resultado negativo.

• Tras comprobar signos, cualquier n negativo se pasa a positivo.

COMPLEMENTO

A 2

PROCEDIMIENTO:

La división se hace restando repetidamente el divisor usando su complemento a 2.

1. Cociente inicial = 0.

2. Se suma el complemento a 2 del divisor al dividendo.

3. Si resultado positivo, se continúa.

4. Si resultado 0 o $-$, la división termina.

5. Cada resta incrementa el cociente en 1.

dividendo
divisor

• Formatos:

• Signo-magnitud: El n° se representa en BCD. El signo se codifica con un grupo adicional de 4 bits (un dígito BCD extra). Se puede usar cualquier combinación BCD no utilizada para $+$ u $-$.

• Complemento a 9: $(C_9(N) = (10^{\text{número de dígitos}} - N) - 1)$

Además se pone bit de signo ($0 \rightarrow +$, $1 \rightarrow -$)

210 $\rightarrow$ 210	0	0010	0001	0000	} RECUERDO A ESTAR EN BCD.
-210 $\rightarrow$ 789	1	0111	0000	1001	

$(C_9(210) = 1000 - 210 - 1 = 789)$

• Complemento a 10: $(C_{10}(N) = (10^n - N))$

• Igual que complemento a 9 pero quitando el -1 de la fórmula.

• Suma BCD

1. Se realiza dígito a dígito una suma binaria de 4 bits.

2. Si hay acarreo $\rightarrow$ se propaga a la suma siguiente.

3. Si resultado $\leq 9 \rightarrow$ válido

4. Si resultado $> 9 \rightarrow$ se suma 6 al resultado / si genera acarreo se suma a siguiente.

ARITMÉTICA

CÓDIGOS

BCD